

FE5: Input and Output

1. Consider the programming models for accessing I/O devices presented on page 7 (and the few following pages) of the I/O slides: *polling*, *using interrupts* and *using Direct Memory Access*. From what you learned on those pages, try to choose the model that would perform better on the following situations:

- i. most of the services asked by an user program to the I/O device are very simple and can be satisfied very quickly;
- ii. the computer's interrupts implementation is rather slow, and a process' context change takes a lot of time;
- iii. most of the I/O operations of a specific device are very slow;
- iv. the computer's bus is very slow;
- v. the CPU operates much faster compared to the speed of the DMA controller.

Of course, the device driver programmer cannot be sure of the specific situations the devices will encounter in a general computer... :-(

2. Regarding the I/O software layer scheme of a typical computer system, presented on p. 16 of the I/O slides, say to which of the layers (one or more) should be assigned the following tasks:

- i. controlling of a user's access permissions to a given block of information;
- ii. calculation of the track, sector and head of a magnetic disk that would correspond to the reading of a given block of information;
- iii. writing of commands to a device controller's registers;
- iv. maintaining a *cache* with the recently accessed blocks of information;
- v. converting binary integers to ASCII characters so that they can be printed.

3. On a Unix system, the I/O devices are accessed almost like files - they are "special" files - and they are usually located in the `/dev` directory. Perform some experiments in order to get some familiarization with these "points of access" to I/O devices. Through a command line interface, do:

- a. `"ls -l /dev/ | sort"`, and observe the type of files shown (remember the "block" and "character" types of devices mentioned on p. 6 of the I/O slides);
- b. `"ls -l /dev/ | sort -k1,1 -k5,5n -k6n"`, and observe that there are several numbers (5th field) that are repeated for several other numbers (6th field), each corresponding to different (special) files, many times with similar names (e.g. `sda1`, `sda2...`). The first numbers are "major" and the following are "minor". Look up their meaning on the web, but a clue is: major numbers represent a group of similar devices; minors identify the device instances within the group; a device driver is written for a group of devices and its code is used by all the devices in the group.
- c. `"sudo cat /dev/input/mouse0"` (or `"cat /dev/input/mice"`), and observe what happens when you move the mouse;
- d. `"man tty"` and `"tty"` and take note of the file name of the terminal where you have given the previous commands (let us say it was, for example, `/dev/pts/2`). Use the file you got in the following experiments; I will use the one of the example. Open a new terminal and in it do:

- "cat > /dev/pts/2", and write some characters; look at your previous terminal (here, /dev/pts/2);
- "ls > /dev/pts/2" and "date > /dev/pts/2" and see what happens on your previous terminal (here, /dev/pts/2).

4. Write a program that allows you to obtain, on a Unix system:

- i. the number of clock interruptions (*clock ticks*) per second;
- ii. the civil date/hour of the "instant zero", reference point used by to mark the beginning of the counting of *real time* seen by the system (*time of day*);
- iii. the size of the system variable used to account for the *real time*.

5. Think of what you appreciated with the program you wrote for exercise 2 and from the knowledge you might have acquired when, in mini-project 1, you tried to get the (real) time of a process and pass it to its children, namely by using one or some of the techniques:

- /proc/stat, with its field "btime", which gives in seconds the booting time of the machine, since the "Epoch" (1970-01-01 00:00:00 +0000, UTC);
- /proc/[pid]/stat, whose word 22 (starttime) gives in clock ticks the instant of time process [pid] has started since the machine's booting time;
- clock_gettime(), with CLOCK_REALTIME, that gives in seconds plus nanoseconds the *real time* since the Epoch.

Try to figure out how the system maintains the real time, out of the three possibilities presented on p. 20 of the I/O slides.

6. A computer's clock system driver is in charge of an important set of administrative tasks. Two of them relate:

- i. to the scheduling of processes and
- ii. to the accounting of processing times (to be charged to each user, for processes they have run).

Explain in some detail how these administrative tasks are performed by the clock system driver.

7. Study the "noecho" sample program designed to demonstrate control over basic text terminal functionality and check the importance of restoring the initial configuration of the terminal.

a. Write a new program that simulates access control of users to a computer system: the program must ask for the user's name and correspondent (secret) access password at the terminal. Enhance your program with the following features:

- have three asterisks (*) shown for each character of the password entered;
- allow the correction of typing mistakes (via the BACKSPACE key).

8. The hard disk repartitioning program used in Unix (fdisk) revealed the following information about a computer's disk:

«Disk /dev/sda: 500.1 GB, 500107862016 bytes; 255 heads, 63 sectors/track, 60801 cylinders».

- a. What is the size of the sector, in *bytes*?
- b. The geometry of the disk appears to be well defined and uniform. Probably, a disk driver could be built capable of controlling even the physical movement and positioning of the reading heads in order to be able to read a specific sector. Is this statement correct?

9. Consider the state of the art of the two current main competing technologies for data storage devices: Solid State Drives (SSD) and Hard Disk Drives (HDD).

- a. Name two advantages of SSD over HDD.
- b. Name two disadvantages of SSD over HDD.
- c. Why is it that Flash NAND storage devices probably take much more time to modify a disk file than to merely append to it?
- d. Random writes to SSD generally perform better (about twice better) than random reads. Try to find out why...

10. The programs that you are going to be asked to write in this problem are intended to exemplify the data flow redirection mechanism, which is characteristic of Unix.

- a. Write a program, *upper*, that displays on the screen (*standard output*) the characters received through the keyboard (*standard input*), but transformed into capital letters. Ensure that your program behaves like a filter and can be inserted between Unix commands connected by '|' (e.g., "shell> ls | upper | sort").
- b. From the previous program, write another program, *upper2*, that performs the same type of operation but that:
 - if it is called with a file name as argument, takes as input the (character) content of that file;
 - if it is called with another file name as second argument, it will send the output to that second file, which should be created if it does not exist. (Hint: the *dup2* service may be useful.)
- c. Create your own version of the Unix's *tee* command!